Introduction to web service

1. Web Service: A Web Service is a technology that allows communication between two different applications over the internet using standard web protocols. It enables Android applications to interact with a remote server to send or receive data.

2. Need of Web Service in Android:

To exchange data between mobile app and server.
To access online databases and services.
To make applications dynamic and real-time.
To integrate different platforms such as Android, Web, and Desktop.

3. Basic Working of Web Service

The Android application sends a request to the web server using HTTP/HTTPS.
The web server processes the request.
The server retrieves data from the database or service.
The server sends a response in JSON or XML format.
The Android app parses the response and displays the data to the user.

4. Types of Web Services:

RESTful Web Service

- Based on HTTP protocol.
- Uses methods like GET, POST, PUT, DELETE.
- Data is usually exchanged in JSON format.
- Lightweight and commonly used in Android applications.

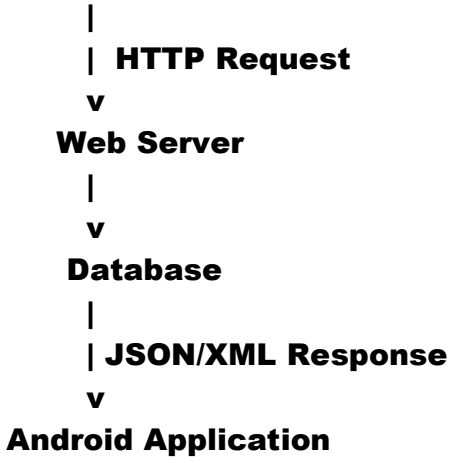
SOAP Web Service

- Uses XML for message communication.
- More secure but complex and heavier.
- Mostly used in enterprise-level applications.

5. Components of Web Service

- Client – The Android application that sends the request.
- Server – Processes the request and returns the response.
- Protocol – HTTP/HTTPS used for communication.
- Data Format – JSON or XML used for data exchange.

6. Web Service Architecture



Platform independent communication.
Supports different programming language.
Easy data sharing between applications.

Android RESTful Web Service Example using Java Servlet

Definition: **A RESTful web service allows an Android application to communicate with a server using HTTP methods such as GET and POST. The server processes the request and returns data in JSON format.**

Java Servlet Example

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/user")
public class UserServlet extends HttpServlet {

 protected void doGet(HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

  response.setContentType("application/json");
  PrintWriter out = response.getWriter();

  out.print("{\"id\":1,\"name\":\"Rahul\"}");
 }
}
```

Architecture / Flow

```
Android Application
        ↓
HTTP Request (GET/POST)
        ↓
Java Servlet (Server)
        ↓
Process Request / Database
        ↓
JSON Response
        ↓
Android Application
```

Android Code to Access REST Service

```
URL url = new URL("http://10.0.2.2:8080/MyProject/user");
HttpURLConnection conn = (HttpURLConnection) url.openConnection();
conn.setRequestMethod("GET");

BufferedReader br = new BufferedReader(
new InputStreamReader(conn.getInputStream()));

String line = br.readLine();
```

Example JSON Response

```
{
 "id":1,
 "name":"Rahul"
}
```

Example : **In an e-commerce Android app, the mobile application sends a request to the server to get product details. The Java Servlet processes the request and returns product information in JSON format, which is displayed in the Android app.**

Conclusion : **Android applications use RESTful web services to communicate with servers.**

Storing Data into External Database

1. External Database

An external database is a database stored on a remote server rather than inside the Android device. Android applications use web services to send data to this database through the internet.

2. Need for External Database

- To store large amounts of application data on a server.
- To allow multiple users to access the same data.
- To provide data backup and security.
- To enable real-time data updates across different devices.

3. Architecture for

Storing Data

Android Application

| HTTP Request (POST)

v

Web Server

|

v

Java Servlet / PHP

|

v

Database (MySQL)

4. Steps to Store Data in External Database

1. Create Database and Table
   Example: MySQL table for storing user data.
2. Create Server-side Script
   A Java Servlet or PHP script is used to receive data from Android.
3. Send Data from Android Application
   Android app sends user input using HTTP POST request.
4. Insert Data into Database
   The server receives the data and runs an SQL INSERT query.

5. Example (Server-side PHP Code)

```
<?php
        $conn = mysqli_connect("localhost","root","","androiddb");
        $name = $_POST['name'];
        $email = $_POST['email'];
        $sql = "INSERT INTO users(name,email) VALUES('$name','$email')";
        mysqli_query($conn,$sql);
        echo "Data Stored Successfully";
?>
```

Explanation

- Android app sends data using HTTP POST request.
- The request reaches the web server.
- Server-side script (Java Servlet / PHP) processes the data.
- Data is stored in the external database (MySQL).

6. Advantages

- Centralized data storage.
- Data can be accessed from multiple devices.
- Easy data management and updates.
- Supports large scale applications.

7. Example

In a student registration Android app, when a student enters details such as name, email, and mobile number, the app sends this data to a server using a web service. The server stores the information in a MySQL external database.

Verifying data in android with external database

1. Data Verification

Data verification in Android with an external database means checking user input data (such as username and password) with the data stored in a remote database using a web service.

2. Purpose of Data Verification

- **To authenticate users before giving access to the application.**
- **To ensure correct and valid data is used.**
- **To improve application security.**
- **To prevent unauthorized access.**

3. Architecture

Android Application
|
| HTTP Request (Login Data)
v
Web Server
|
v
Server Script (Servlet / PHP)
|
v
Database (MySQL)
|
| Verification Result
v
Android Application

4. Working Process

- **User enters username and password in the Android app.**
- **The Android application sends this data to the server using HTTP POST request.**
- **The server-side program (Java Servlet / PHP) receives the data.**
- **The server checks the data with the database records.**
- **The server sends a response (success or failure) back to the Android app.**

5. Example of Server-side Verification (PHP)

```
<?php
   $conn = mysqli_connect("localhost","root","","androiddb");
   $username = $_POST['username'];
   $password = $_POST['password'];
   $sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
   $result = mysqli_query($conn,$sql);
   if(mysqli_num_rows($result) > 0){
     echo "Login Successful";
   }else{
      echo "Invalid Username or Password";
   }
?>
```

6. Android Application Process

- **Android app sends login data to the server.**
- **Receives the response from the server.**
- **If response is valid, the user is allowed to access the application.**
- **If response is invalid, an error message is displayed.**

7. Example

In an Android login application, the user enters a username and password.The app sends these details to the server, where the database checks the credentials. If the details match, the user is successfully logged in.

XML Parsing SAX

1. XML Parsing

XML Parsing is the process of reading XML data and extracting useful information from it. In Android applications, XML parsing is used to retrieve data received from web services or remote servers.

2. SAX Parser

SAX (Simple API for XML) is an event-driven XML parsing method. It reads the XML document sequentially from top to bottom and triggers events when it encounters XML tags.

3. Features of SAX Parser

- Event-based parsing technique.
- Reads XML line by line.
- Does not store the entire XML file in memory.
- Faster and more memory efficient for large XML files.

4. Working of SAX Parser

- The SAX parser starts reading the XML document.
- When it finds the start tag, it triggers the startElement() event.
- When it reads the data inside the tag, it triggers the characters() event.
- When it finds the end tag, it triggers the endElement() event.

5. XML Example

```
<students>
   <student>
      <name>Rahul</name>
      <city>Ahmedabad</city>
   </student>
</students>
```

6. SAX Parser Implementation in Android

```
SAXParserFactory factory = SAXParserFactory.newInstance();
SAXParser saxParser = factory.newSAXParser();

DefaultHandler handler = new DefaultHandler() {
   public void startElement(String uri, String localName,
       String qName, Attributes attributes) {
   }
   public void characters(char ch[], int start, int length) {
   }
   public void endElement(String uri, String localName,
       String qName) {
   }
};
```

7. Advantages of SAX Parser

- Uses less memory compared to other parsers.
- Suitable for large XML files.
- Faster because it processes data sequentially.

8. Example

In an Android news application, the app receives news data in XML format from a web server. The SAX parser reads the XML file and extracts information such as title, description, and date, which is then displayed in the Android application.

XML Parsing DOM

1. DOM Parser

DOM (Document Object Model) parser is an XML parsing method that loads the entire XML document into memory and represents it as a tree structure. Android applications use DOM parser to read and manipulate XML data easily.

2. Features of DOM Parser

- **Loads the complete XML file into memory.**
- **Represents XML as a tree structure of nodes.**
- **Allows random access to elements in the XML document.**
- **Easy to read and modify XML data.**

3. Working of DOM Parser

- **The parser first loads the entire XML document.**
- **It creates a tree structure with nodes.**
- **Each XML tag is treated as a node.**
- **The application can access elements using node methods.**

4. XML Example

```
<students>
   <student>
      <name>Rahul</name>
      <city>Ahmedabad</city>
   </student>
</students>
```

5. DOM Parsing Example (Android)

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(inputStream);
NodeList list = doc.getElementsByTagName("student");
```

6. Example Application

In an Android student information app, XML data received from the server contains student details. The DOM parser reads the XML document and extracts information such as name and city to display in the application.

SOAP vs RESTful Web Service

Feature	SOAP Web Service	RESTful Web Service
Protocol	Uses a strict protocol called SOAP (Simple Object Access Protocol).	Uses standard HTTP protocol for communication.
Data Format	Only supports XML format for sending and receiving data.	Supports multiple formats such as JSON, XML, and plain text.
Performance	Slower because XML messages are heavy and require more processing.	Faster and lightweight, especially when using JSON.
Complexity	More complex to implement and requires strict standards.	Simple to develop and easier to integrate with web and mobile apps.
Usage	Mostly used in enterprise-level applications requiring high security and reliability.	Widely used in modern web services, mobile applications, and APIs.

XML Pull Parser

XML Pull Parser is a stream-based XML parsing technique used in Android to read XML data. It allows the application to manually control the parsing process by pulling events from the XML document.

2. Features of XML Pull Parser

- **Lightweight and efficient parser.**
- **Provides better performance than DOM parser.**
- **Uses less memory because it reads XML step by step.**
- **The application controls the parsing process.**

3. Working of XML Pull Parser

- **The parser reads the XML file sequentially.**
- **It generates events such as START_TAG, TEXT, and END_TAG.**
- **The application processes these events and extracts required data.**

4. XML Example

```
<students>
   <student>
      <name>Rahul</name>
      <city>Ahmedabad</city>
   </student>
</students>
```

5. Example Code (Android)

```
XmlPullParserFactory factory = XmlPullParserFactory.newInstance();
XmlPullParser parser = factory.newPullParser();

parser.setInput(inputStream, null);

int eventType = parser.getEventType();

while (eventType != XmlPullParser.END_DOCUMENT) {
    if (eventType == XmlPullParser.START_TAG) {
        String tagName = parser.getName();
    }
    eventType = parser.next();
}
```

6. Example Application

In an Android news or weather application, XML data received from a web service is parsed using XML Pull Parser to extract information such as title, temperature, or description and display it in the app.

JSON Parsing

1. JSON:

JSON (JavaScript Object Notation) is a lightweight data interchange format used for transmitting data between a server and an Android application. It is easy to read, write, and parse.

2. JSON Parsing:

JSON Parsing is the process of reading JSON data and extracting required information so that it can be used in an Android application.

3. Features of JSON:

- **Lightweight and fast data format.**
- **Easy to read and write.**
- **Uses key–value pair structure.**
- **Widely used in RESTful web services.**

4. Structure of JSON

Example JSON data:

```
{
  "student": {
    "name": "Rahul",
    "city": "Ahmedabad",
    "age": 22
  }
}
```

Explanation

- **student → JSON Object**
- **name, city, age → Keys**
- **Rahul, Ahmedabad, 22 → Values**

5. Steps for JSON Parsing in Android

- **Receive JSON data from a web service or server.**
- **Convert the response into a JSON object or JSON array.**
- **Extract required values using keys.**
- **Display the data in the Android user interface.**

6. Example Code for JSON Parsing

```
JSONObject jsonObject = new JSONObject(jsonString);

String name = jsonObject.getString("name");
String city = jsonObject.getString("city");
int age = jsonObject.getInt("age");
```

Explanation

- **JSONObject is used to read JSON data.**
- **getString() and getInt() methods extract values from JSON.**

7. Advantages of JSON Parsing

- **Faster than XML parsing.**
- **Uses less bandwidth.**
- **Easy to integrate with RESTful web services.**
- **Supported by most programming languages.**

8. Example Application

In an Android weather application, the app sends a request to a web service and receives weather information in JSON format. JSON parsing is used to extract data such as temperature, humidity, and weather condition, which is then displayed in the mobile application.

Integrating Social Networking using HTTP

1. Social Networking Integration

Integrating social networking in Android means connecting an Android application with social media platforms such as Facebook, Twitter, or Instagram using HTTP protocol to share or retrieve information.

2. HTTP Protocol

HTTP (Hyper Text Transfer Protocol) is used to send requests and receive responses between the Android application and social networking servers over the internet.

3. Purpose of Social Network Integration

- To allow users to share content directly from the application.
- To enable login using social media accounts.
- To fetch user profile information and posts.
- To improve user interaction and application popularity.

4. Working Process

- Android application sends an HTTP request to the social networking server.
- The server processes the request.
- The server returns a response in JSON or XML format.
- The Android application processes the data and displays it.

5. HTTP Methods Used

- **GET** – used to retrieve data from the social network server.
- **POST** – used to send data such as posts or messages.
- **PUT** – used to update information.
- **DELETE** – used to remove data.

6. Basic Example (HTTP Connection in Android)

```
URL url = new URL("https://api.socialnetwork.com/user");
HttpURLConnection connection = (HttpURLConnection)
url.openConnection();
connection.setRequestMethod("GET");
```

Explanation

- URL defines the API address of the social network.
- HttpURLConnection creates a connection to the server.
- GET method retrieves user data from the server.

7. Advantages

- Easy sharing of content from the application.
- Improves user engagement and connectivity.
- Allows applications to access social media data.
- Helps in authentication using social accounts.

8. Example

In a photo sharing Android application, users can upload images and directly share them on Facebook or Twitter using HTTP requests. The application sends the image and message to the social media server, which publishes the post on the user's account.

Monitoring and managing Internet connectivity

1. Internet Connectivity in Android

Internet connectivity in Android refers to the ability of a device to connect to the internet using Wi-Fi, mobile data, or other network technologies. Android provides APIs to monitor and manage network connectivity.

2. Need for Monitoring Internet Connectivity

- **To check whether the device is connected to the internet.**
- **To detect network changes such as switching from Wi-Fi to mobile data.**
- **To prevent application errors when the internet is not available.**
- **To improve performance of network-based applications.**

3. ConnectivityManager Class

Android provides the ConnectivityManager class to monitor and manage internet connectivity.

It provides information about active networks and network status.

4. Checking Internet Connectivity

Example Code:

```
ConnectivityManager cm = (ConnectivityManager)
getSystemService(Context.CONNECTIVITY_SERVICE);
NetworkInfo networkInfo = cm.getActiveNetworkInfo();
if(networkInfo != null && networkInfo.isConnected()) {
    // Internet connection available
} else {
    // No internet connection
}
```

Explanation

- **ConnectivityManager is used to manage network connections.**
- **getActiveNetworkInfo() returns the current active network.**
- **isConnected() checks whether the device is connected to the internet.**

5. Monitoring Network Changes

Android applications can use BroadcastReceiver to detect changes in internet connectivity such as:

- **Wi-Fi turned ON or OFF**
- **Mobile data enabled or disabled**
- **Network connection lost or restored**

6. Managing Network Connections

Android applications can manage connectivity by:

- **Checking available network types (Wi-Fi or mobile data).**
- **Handling network interruptions during data transfer.**
- **Adjusting application behavior based on network availability.**

7. Advantages

- **Helps applications detect network availability.**
- **Prevents network-related failures.**
- **Improves user experience in online applications.**

8. Example

In an Android online video streaming application, the app checks internet connectivity before loading video content. If there is no internet connection, the application displays a "No Internet Connection" message to the user.

Managing active connections

1. Active Connections

Active connections refer to the current network connections available on an Android device, such as Wi-Fi, mobile data, or other network interfaces that allow the device to access the internet.

2. Need for Managing Active Connections

- **To check the status of current network connections.**
- **To determine which network is currently active.**
- **To manage network usage efficiently.**
- **To improve application performance and reliability.**

3. Connectivity Manager

Android uses the ConnectivityManager class to manage and monitor active network connections. It provides methods to retrieve information about the current network state.

4. Checking Active Network Connection

Example Code:

```
ConnectivityManager cm = (ConnectivityManager)
getSystemService(Context.CONNECTIVITY_SERVICE);
NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
if(activeNetwork != null && activeNetwork.isConnected()){
    String type = activeNetwork.getTypeName();
}
```

Explanation

- **ConnectivityManager is used to access network connectivity information.**
- **getActiveNetworkInfo() returns the currently active network.**
- **isConnected() checks whether the network is connected.**
- **getTypeName() identifies the network type such as Wi-Fi or Mobile Data.**

5. Types of Active Connections

- **Wi-Fi Connection – Internet connection through wireless network.**
- **Mobile Data Connection – Internet using cellular network.**
- **VPN Connection – Secure connection through virtual private network.**

6. Managing Network State Android applications can:

- **Check if Wi-Fi or mobile data is active.**
- **Monitor network changes.**
- **Handle network interruptions during data transfer.**

7. Advantages

- **Helps applications use the best available network.**
- **Prevents data transfer errors.**
- **Improves network efficiency and performance.**

8. Example

In an Android video streaming application, the app checks the active connection before streaming. If Wi-Fi is available, the application streams high-quality video; otherwise, it uses mobile data with lower quality to reduce data usage.

Managing Wi-Fi Networks

1. Wi-Fi in Android

Wi-Fi is a wireless network technology that allows Android devices to connect to the internet through a wireless router or access point.

2. WiFiManager Class

Android provides the WifiManager class to manage Wi-Fi connectivity. It allows applications to enable, disable, scan, and connect to Wi-Fi networks.

3. Permissions Required

To manage Wi-Fi networks, the following permission is required in the Android manifest file:

```
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
```

4. Enabling and Disabling Wi-Fi

Example Code:

```
WifiManager wifiManager = (WifiManager)
getApplicationContext().getSystemService(Context.WIFI_SERVICE);

wifiManager.setWifiEnabled(true);   // Enable Wi-Fi
wifiManager.setWifiEnabled(false);  // Disable Wi-Fi
```

Explanation

- WifiManager is used to control Wi-Fi operations.
- setWifiEnabled(true) turns on Wi-Fi.
- setWifiEnabled(false) turns off Wi-Fi.

5. Scanning Available Wi-Fi Networks:

Android applications can scan nearby Wi-Fi networks using the startScan() method.

Example:

```
wifiManager.startScan();
List<ScanResult> wifiList = wifiManager.getScanResults();
```

6. Connecting to a Wi-Fi Network

The application can configure a Wi-Fi connection by creating a WifiConfiguration object and connecting to the desired network.

7. Advantages

- Allows applications to manage wireless connectivity.
- Helps detect and connect to available Wi-Fi networks.
- Provides better internet performance compared to mobile data.

8. Example

In an Android smart home application, the app scans nearby Wi-Fi networks and connects the device to the home router so that the user can control smart devices through the internet.

Controlling Local Bluetooth Device

1. Bluetooth in Android

Bluetooth is a wireless communication technology used for short-range data exchange between devices such as smartphones, laptops, and headphones. Android provides APIs to control the local Bluetooth device.

2. BluetoothAdapter Class

Android uses the BluetoothAdapter class to manage Bluetooth functions.

It represents the local Bluetooth adapter and allows applications to enable, disable, and control Bluetooth operations.

3. Checking Bluetooth Availability

Example Code:

```
BluetoothAdapter bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();
if(bluetoothAdapter == null){
    // Device does not support Bluetooth
}
```

Explanation

- **getDefaultAdapter() returns the local Bluetooth adapter.**
- **If it returns null, the device does not support Bluetooth.**

4. Enabling and Disabling Bluetooth

Example Code:

```
if(!bluetoothAdapter.isEnabled()){
   bluetoothAdapter.enable();   // Enable Bluetooth
}
bluetoothAdapter.disable();      // Disable Bluetooth
```

Explanation

- **isEnabled() checks whether Bluetooth is ON or OFF.**
- **enable() turns Bluetooth ON.**
- **disable() turns Bluetooth OFF.**

5. Getting Device Information

Example Code:

```
String deviceName = bluetoothAdapter.getName();
String deviceAddress = bluetoothAdapter.getAddress();
```

Explanation

- **getName() returns the name of the Bluetooth device.**
- **getAddress() returns the unique Bluetooth MAC address.**

6. Bluetooth Permissions

Required permissions in Android Manifest:wha

```
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
```

7. Advantages

- **Enables wireless communication between nearby devices.**
- **Useful for data transfer and device connectivity.**
- **Low power consumption compared to other wireless technologies.**

8. Example

In an Android file sharing application, Bluetooth is used to enable the device's Bluetooth adapter, detect nearby devices, and transfer files between two Android smartphones.

Discovering and Bonding with Bluetooth Devices

1. Definition: Bluetooth in Android allows devices to communicate wirelessly over short distances.
Discovering means searching for nearby Bluetooth devices, and Bonding means creating a trusted connection between two devices.

---

2. Discovering Bluetooth Devices: Device discovery is the process of searching for nearby Bluetooth devices.
Steps

1. Get Bluetooth Adapter.
2. Enable Bluetooth on the device.
3. Start device discovery.
4. Receive discovered devices using BroadcastReceiver.

Example Code

```
BluetoothAdapter bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();

if(!bluetoothAdapter.isEnabled())
{
    bluetoothAdapter.enable();
}

bluetoothAdapter.startDiscovery();
```

Explanation

- BluetoothAdapter → Represents the Bluetooth hardware.
- enable() → Turns on Bluetooth.
- startDiscovery() → Starts scanning nearby devices.

---

3. Bonding with Bluetooth Devices: Bonding creates a paired connection between two Bluetooth devices so they can communicate securely.
Steps

1. Discover nearby devices.
2. Select the device to connect.
3. Request pairing (bonding).
4. Devices exchange security keys.

Example Code

```
BluetoothDevice device = bluetoothAdapter.getRemoteDevice(address);
device.createBond();
```

Explanation

- BluetoothDevice → Represents the remote device.
- createBond() → Initiates pairing between devices.

---

4. Example: When a user connects an Android phone to a Bluetooth headset or speaker, the phone first discovers nearby devices and then bonds (pairs) with the selected device to allow communication.

---

5. Conclusion
In Android, Bluetooth discovery is used to find nearby devices, while bonding is used to establish a secure paired connection between devices for communication and data transfer.

Managing Bluetooth connections

Definition: Managing Bluetooth connections in Android refers to establishing, maintaining, and closing communication between two Bluetooth devices so that they can exchange data reliably.

---

1. Steps to Manage Bluetooth Connections

1. Enable Bluetooth on the device.
2. Discover nearby Bluetooth devices.
3. Pair (bond) with the selected device.
4. Establish connection using Bluetooth sockets.
5. Transfer data between devices.
6. Close the connection when communication is finished.

---

2. Establishing Bluetooth Connection

After pairing, a connection is created using BluetoothSocket.

Example Code

```
BluetoothDevice device = bluetoothAdapter.getRemoteDevice(address);

BluetoothSocket socket =
device.createRfcommSocketToServiceRecord(UUID);

socket.connect();
```

Explanation

- **BluetoothDevice → Represents the remote Bluetooth device.**
- **BluetoothSocket → Used to create a communication channel.**
- **connect() → Establishes the connection between devices.**

---

3. Sending and Receiving Data: Once the connection is established, data can be transferred using input and output streams.

Example Code

```
OutputStream out = socket.getOutputStream();
InputStream in = socket.getInputStream();
```

---

4. Closing Bluetooth Connection

After communication is completed, the connection should be closed.

socket.close();

This releases system resources and stops communication.

---

5. Example: When an Android phone is connected to a Bluetooth speaker, the phone manages the Bluetooth connection by pairing with the speaker, establishing a socket connection, transmitting audio data, and disconnecting when the device is turned off.

Conclusion: Managing Bluetooth connections in Android involves pairing devices, creating socket connections, transferring data, and properly closing the connection to ensure efficient communication.

Communicating with Bluetooth

Definition: **Bluetooth communication in Android allows two paired devices to exchange data wirelessly using Bluetooth sockets and input/output streams.**

---

1. Requirements for Bluetooth Communication

- **Bluetooth must be enabled on both devices.**
- **Devices must be paired (bonded).**
- **A BluetoothSocket connection must be established.**

---

2. Steps for Bluetooth Communication

1. **Enable Bluetooth on the device.**
2. **Discover nearby Bluetooth devices.**
3. **Pair with the required device.**
4. **Establish a connection using BluetoothSocket.**
5. **Send and receive data using input and output streams.**

---

3. Sending Data Example

```
BluetoothSocket socket =
device.createRfcommSocketToServiceRecord(UUID);
socket.connect();

OutputStream outputStream = socket.getOutputStream();
outputStream.write("Hello".getBytes());
```

Explanation

- **BluetoothSocket creates a communication channel.**
- **connect() establishes the connection.**
- **OutputStream sends data to the connected device.**

---

4. Receiving Data Example

```
InputStream inputStream = socket.getInputStream();
byte[] buffer = new byte[1024];
inputStream.read(buffer);
```

Explanation

- **InputStream receives data from the connected device.**
- **read() reads the incoming data.**

---

5. Example

When an Android phone sends a file to another phone using Bluetooth, the devices first connect through Bluetooth sockets. The sender uses OutputStream to send data, and the receiver uses InputStream to receive the data

---

Conclusion

Bluetooth communication in Android is achieved by creating a BluetoothSocket connection and using input and output streams to send and receive data between paired devices.